

Architectural Viability of the APEX AI Race Engineer: A Comparative Analysis of Foundation Models

The development of autonomous and semi-autonomous adaptive driving systems introduces unprecedented architectural demands, requiring a computational framework capable of processing high-frequency telemetry, enforcing non-negotiable physical constraints, and parsing highly idiosyncratic regulatory frameworks in real time. The APEX system—developed as an AI race engineer for adaptive racing drivers and submitted to the IBM SkillsBuild AI Builders Challenge (May 2026)—proposes an innovative tri-layered architectural pattern. This pattern is defined as a "frozen pretrained foundation model plus differentiable projection plus text audit." Within this paradigm, a foundational time-series model generates continuous forecasts for lap-time, speed, and acceleration. These statistical forecasts are subsequently passed through a differentiable convex quadratic programming (QP) layer, which strictly projects the predicted trajectory onto a feasible-physics envelope governed by friction limits and forward-Euler kinematics. Finally, a specialized textual audit layer examines the violation logs to ensure the physical limits align with specific regulatory de-homologations granted to adaptive drivers.

This comprehensive technical report provides an exhaustive evaluation of this specific forecast-project-audit pattern, focusing predominantly on evaluating the IBM Granite ecosystem against competing state-of-the-art foundation models. The analysis systematically unpacks the mathematical, architectural, and operational nuances of the IBM Granite TimeSeries suite (TTM r2.1, FlowState 9.1M, TSPulse 1M) and Granite Guardian 4.1 8B against leading alternatives, including Amazon Chronos-2, Google TimesFM, Salesforce Moirai, Meta Llama 3.3, Mistral Large 2, and frontier models from Anthropic and Google.

The Telemetry Generation Layer: Time-Series Foundation Models

The primary ingestion and forecasting stage of the APEX architecture necessitates a time-series foundation model capable of processing multi-channel telemetry streams. The operational constraint dictating the selection of this model is the strict adherence to a frozen backbone with minimal downstream modification, requiring exceptional sample efficiency and an architecturally lightweight profile suitable for edge deployment.

The IBM Granite TimeSeries Ecosystem

The IBM Granite TimeSeries family approaches the forecasting challenge through extreme parameter efficiency, predominantly relying on patch-based architectures and specialized pretraining strategies that circumvent the computational bloat associated with large language models.¹

The TinyTimeMixer (TTM) r2.1 model represents a significant evolution in compact forecasting.

Pretrained on approximately one billion samples—a substantial increase from the 250 million samples utilized in the r1 variant—TTM r2.1 introduces the concept of the "tiny" pretrained model for multivariate forecasting, operating with parameter counts starting at merely one million.¹ The core architectural philosophy of TTM is channel independence during pretraining. Under this paradigm, the frozen backbone learns a universal, common dynamics model across all incoming data channels without explicitly computing the mathematical interactions between them.³ While highly efficient, automotive telemetry involves deeply correlated variables, such as the direct relationship between throttle application, steering angle, and lateral acceleration. To capture these complex cross-channel dependencies without unfreezing the massive pretraining weights, TTM r2.1 enables a specialized channel-mixing decoder during the fine-tuning phase.³ This targeted fine-tuning strategy is extraordinarily sample-efficient, requiring only five percent of the target dataset to lift the metric and achieve competitive multivariate forecasting, making it highly adaptable for newly integrated vehicle sensors.¹ Complementing TTM, the FlowState 9.1M model introduces timescale invariance to the Granite suite. Telemetry systems often suffer from irregular sampling rates or varying frequencies across different sensor arrays (e.g., suspension potentiometers sampling at 1000Hz while GPS samples at 10Hz). FlowState resolves this through a sophisticated architecture that integrates a State Space Model (SSM) encoder with a functional basis decoder.² This mechanism transitions the input data into a timescale-invariant coefficient space, enabling continuous forecasting that adapts to any underlying sampling rate.² Consequently, FlowState can be trained on telemetry recorded at one temporal resolution and execute inference at an entirely different resolution, delivering state-of-the-art zero-shot accuracy across multiple domains and effectively neutralizing the irregularity of edge-sensor data streams.² At the most compact end of the spectrum, the TSPulse 1M model operates with exactly one million parameters, utilizing a novel dual-space masked reconstruction framework.² Unlike traditional models that process signals solely in the temporal domain, TSPulse jointly learns from both the time and frequency domains, unifying these complementary signals into a shared embedding space.² Through a dual-embedding disentanglement mechanism, TSPulse systematically separates fine-grained localized patterns from broad global semantics.² Despite its microscopic parameter footprint, this disentanglement property yields massive zero-shot performance gains, including a fifty percent improvement in data imputation and a twenty percent improvement in anomaly detection across the TSB-AD leaderboard, making it highly resilient to dropped telemetry packets.⁸

Natural Language Adaptations: Amazon Chronos-2

Amazon Chronos-2 diverges entirely from the patching mechanisms of Granite, instead adapting standard causal language model architectures for time-series data.⁹ The Chronos pipeline operates by normalizing input targets and covariates through a robust scaling scheme, followed by the transformation of continuous time-series values into a discrete sequence of tokens via quantization.⁹ These tokens are then processed by an architecture inspired by the T5 encoder, training the model using standard cross-entropy loss.¹⁰

The defining competitive advantage of Chronos-2 is its implementation of a sophisticated group attention mechanism.¹¹ This mechanism facilitates highly efficient zero-shot in-context learning (ICL) by sharing information across multiple time series within a designated group, which can represent different variates of a multivariate series or dynamic covariates.¹¹ As a result, Chronos-2 inherently processes complex multivariate relationships without the necessity of the channel-mixing decoder fine-tuning required by IBM Granite TTM.³ Furthermore, Chronos-2 natively produces multi-step-ahead quantile forecasts, providing direct probabilistic bounds that are invaluable for estimating epistemic uncertainty in high-speed kinematic predictions.¹¹

Patch-Based Transformers: Google TimesFM and Salesforce Moirai

Google's TimesFM 2.5 occupies the middle ground regarding parameter size, operating as a 200-million parameter, decoder-only foundation model.¹³ The architecture relies on segmenting time-series sequences into discrete patches, typically defaulting to a patch length of 32, and processing them through a deep Transformer decoder stack featuring up to 50 hidden layers and a hidden size dimension of 1280.¹⁵ TimesFM accepts extensive context lengths—up to 16,384 tokens in version 2.5—allowing the model to analyze prolonged historical telemetry to inform its prediction horizon.¹⁵ Like Chronos, TimesFM provides both point and quantile forecasts out of the box, preserving critical forecasting behaviors natively.¹⁴ Salesforce approaches the foundation model paradigm through Moirai and Lag-Llama. Moirai 2.0 is trained on the massive Large-scale Open Time Series Archive (LOTSAs), a dataset comprising 27 billion observations across nine distinct domains.¹⁷ To manage the extreme heterogeneity of arbitrary time-series data, Moirai utilizes multiple input and output projection layers designed specifically to handle varying data frequencies.¹⁷ The architecture maps the output representations of these patches directly into the parameters of a mixture distribution, enabling universal forecasting capabilities.¹⁷ Conversely, the Lag-Llama framework explicitly adapts the LLaMA decoder-only transformer architecture for univariate probabilistic forecasting.¹⁸ The final layer of the Lag-Llama network acts as a specialized distribution head, projecting the learned feature representations directly into the mathematical parameters—mean, scale, and degrees of freedom—of a Student's t-distribution.¹⁹ This specific probabilistic output is highly robust to heavy-tailed noise, allowing the model to estimate uncertainty directly without relying on external post-hoc conformal prediction pipelines.²⁰

LLM Reprogramming Frameworks

Rather than constructing purpose-built architectures from the ground up, frameworks such as Time-LLM, AutoTimes, and GPT4TS attempt to repurpose existing, frozen Large Language Models (LLMs) for forecasting tasks.²¹ The Time-LLM reprogramming framework casts time-series analysis as a derivative language task.²¹ The methodology involves taking the input time-series data, stripping it of its raw numerical context, and reprogramming it into text-prototype representations that align more naturally with the pretrained modalities of the LLM.²¹ To further augment the reasoning capacity of the frozen LLM, Time-LLM introduces a

"Prompt-as-Prefix" (PaP) mechanism, which enriches the input context and explicitly directs the transformation of the reprogrammed input patches into eventual forecasts.²²

While this methodology prevents the need to alter the internal layers of the backbone model—maintaining strict adherence to the definition of model reprogramming²³—it inherits the catastrophic latency and memory overhead associated with billion-parameter architectures. AutoTimes attempts to mitigate the adaptation cost by introducing minimal trainable parameters while keeping all intermediate layers of the LLM frozen, allowing textual timestamps to be integrated without excessive runtime overhead.²⁴ Nonetheless, deploying a 7-billion parameter language model solely to predict localized vehicle acceleration trajectories violates the latency thresholds required for real-time edge processing, rendering these reprogramming frameworks fundamentally non-viable for the APEX race engineer architecture.²⁴

Foundation Model Architecture	Parameter Scale	Pretraining Methodology	Multivariate Processing Strategy	Forecasting Output Type
IBM Granite TTM r2.1	~1 Million	Channel Independent	Decoder Channel-Mixing (Fine-tune)	Point Forecasts
IBM FlowState 9.1M	9.1 Million	State Space Model (SSM)	Timescale Invariant Coefficient Space	Point Forecasts
IBM TSPulse 1M	1 Million	Dual-Space Masked Reconstruction	Dual-Embedding Disentanglement	Point Forecasts
Amazon Chronos-2	T5 Equivalent	Tokenization & Quantization	Group Attention In-Context Learning	Quantile Forecasts
Google TimesFM 2.5	200 Million	Patch-based Decoder-only	Native Context Window	Point and Quantile
Salesforce Moirai 2.0	Small/Base/Large	LOTSa Corpus (27B points)	Multiple Patch Size Projection	Mixture Distribution

			Layers	
Lag-Llama	LLaMA Base	Decoder-only Transformer	Distribution Head Projection	Student's t-distribution
Time-LLM	>7 Billion	Text-Prototype Reprogramming	Prompt-as-Prefix (PaP)	Point Forecasts

The Differentiable Projection Layer: Enforcing Kinematic Boundaries

The second fundamental stage of the APEX architecture is the differentiable projection layer. Time-series foundation models, operating purely on statistical correlations, frequently generate predictive trajectories that are statistically probable but physically impossible. To bridge the gap between statistical inference and vehicular reality, APEX employs a differentiable convex quadratic programming (QP) projection layer.

This projection algorithm strictly maps the raw, unconstrained statistical forecast generated by the Granite models onto a mathematically rigorous, feasible-physics envelope. For the APEX system, this envelope is defined by three intersecting constraints. The first is the friction ellipse, a non-linear constraint that restricts the combined vector sum of longitudinal acceleration (braking and accelerating) and lateral acceleration (cornering) to the ultimate physical grip limit of the vehicle's tires.²⁵ The second constraint relies on a forward-Euler kinematic step, effectively enforcing the continuous, step-by-step mathematical integration of the vehicle's position, velocity, and acceleration over time, ensuring that the vehicle does not teleport across the coordinate space. The third constraint is the brake-throttle simultaneity gate, a dynamically parameterized boundary that regulates the overlap between throttle and brake application, specifically calibrated for the unique hand-control adaptations utilized by disabled drivers.

Implicit Differentiation and Optimization Solvers

To ensure that the upstream Granite forecasting model learns to respect these downstream physical constraints during the fine-tuning phase, the projection operator must be fully differentiable.²⁶ The APEX architecture relies on frameworks such as cvxpylayers to embed the complex convex optimization problem directly into the neural network architecture as a distinct, differentiable layer.²⁶

The critical challenge in differentiable optimization is propagating gradients backward through the iterative steps of an optimization solver.²⁹ Attempting to unroll the forward operations of the solver and applying standard automatic differentiation (autodiff) results in unacceptable

computational complexity that scales linearly with the number of optimization steps, heavily penalizing GPU memory.²⁸ To circumvent this, cvxpylayers and similar frameworks utilize implicit algorithmic differentiation.²⁸

Implicit differentiation bypasses the iterative steps entirely. Instead, it relies on the mathematical properties of the optimal solution. By analyzing the stationary Karush-Kuhn-Tucker (KKT) optimality conditions at the solution point, the framework constructs an expression of the Jacobian.²⁸ The computation of the gradient via implicit differentiation involves the resolution of a large linear system of equations.³¹ The blockwise structure of the optimal solution simplifies this differentiation process; for instance, the Jacobian of an isotonic projection inherently assumes a block-diagonal structure, allowing for uniform gradient propagation within specific blocks and ensuring an operation complexity that remains linear with respect to the input size.³² This enables the projection layer to strictly enforce the physical constraints in the forward pass while simultaneously inducing policy gradients that train the neural network to become inherently cognizant of the physical limitations during the backward pass.²⁶

Numerical Instability and Gradient Pathology

Despite the theoretical elegance of implicit differentiation, the practical implementation of cvxpylayers within highly dynamic environments presents severe numerical and computational challenges. The framework relies on underlying numerical conic solvers, such as the Splitting Conic Solver (SCS) or Clarabel, to find the optimal solution in the forward pass.³³ When the operational constraints become too tight—such as simultaneously maximizing velocity while adhering strictly to the extreme boundary of the friction ellipse and the brake-throttle gate—these solvers frequently fail to converge within the allotted iterations.³⁴

These solver failures manifest as SolverError exceptions, returning statuses such as Infeasible or Unbounded (inaccurate - reached max_iters).³⁴ When the forward solver fails to find a precise optimal point, the subsequent linear system constructed for the KKT conditions becomes ill-posed, resulting in the calculation of NaN (Not a Number) gradients during the backward pass.³⁴ The propagation of NaN gradients instantly corrupts the weights of the neural network, collapsing the entire training process.

Furthermore, even when the solver successfully converges, the application of implicit algorithmic differentiation is exceptionally sensitive to the accumulation of local rounding errors.³¹ When implicit differentiation is applied to non-invertible boundary conditions without strict regulatory hypotheses, it produces extremely pathological gradient dynamics.³⁶ Research has demonstrated that performing gradient ascent with implicit differentiation on highly constrained polynomials can generate chaotic pathways that perfectly resemble the Lorenz attractor, starkly contrasting the expected convergent pathways of conventional optimization.³⁶ Consequently, stabilizing the differentiable projection layer within the APEX architecture necessitates aggressive mitigation strategies, including extensive matrix scaling, normalizing the data structures, loosening the solver tolerance epsilons, and implementing rigorous gradient clipping to prevent the chaotic amplification of rounding errors.³³

The Textual Audit Layer: Parsing the Regulatory Niche

While the differentiable projection layer mathematically enforces the laws of physics, the third layer of the APEX architecture enforces the laws of the governing sporting body. In standard motorsport, vehicles conform to a strict homologation document that defines the exact physical parameters permitted by the regulations. However, the adaptive racing niche operates on a fundamentally different premise.

Disabled drivers, operating under the purview of the Fédération Internationale de l'Automobile (FIA) Disability and Accessibility Commission, are granted specialized Certificates of Adaptation.³⁷ This certificate acts as a formal de-homologation, explicitly permitting modifications that violate standard regulations, such as the implementation of heavily customized roll-bar structures to aid extraction, the relocation of fuel tanks to accommodate structural changes, or the installation of bespoke hand-operated throttle and brake mechanisms.³⁷

The APEX architecture generates a highly detailed violation log whenever the output of the forecasting model clashes with the physical projection layer. To determine whether a detected physical violation is an illegal maneuver or a legally permitted adaptation under a specific driver's FIA certificate, APEX employs a large language model to perform a textual audit of the violation log against the parsed regulatory input, ensuring absolute engine-agnostic byte-equality across both V1 NumPy and V2 cvxpylayers projectors.²⁸

IBM Granite Guardian 4.1 8B and the BYOC Paradigm

To execute this highly specialized audit, the architecture utilizes the IBM Granite Guardian 4.1 8B model. Unlike generalized language models tuned for conversational fluency, the Granite Guardian family is explicitly engineered to judge whether the input and output responses of an underlying system meet highly specific, mathematically rigid criteria.⁴²

The 4.1 release introduces a sophisticated Bring-Your-Own-Criteria (BYOC) infrastructure.⁴² This infrastructure bypasses standard pre-baked safety taxonomies, allowing developers to define arbitrary evaluation rules expressed entirely in natural language, such as validating complex domain constraints regarding D-050 directives or verifying compliance with multi-part engineering requirements parsed from the FIA dossier.⁴²

The operational superiority of Granite Guardian 4.1 for this specific task lies in its rigid prompt format template. The evaluation is passed via a user message containing a clearly defined <guardian> block.⁴² This block is segmented into three strictly enforced components. The first is the Mode Selector, where the architecture can specify the GUARDIAN_JUDGE_THINK instruction, prompting the model to explicitly reason through the complex engineering evidence step-by-step and wrap its logic in <think> tags, or the GUARDIAN_JUDGE_NOTHINK instruction, forcing the model to skip the chain of thought and output a score instantly for minimum latency.⁴² The second component is the ### Criteria: section, where the raw, unstructured text of the FIA Certificate of Adaptation is injected.⁴² The final component is the ### Scoring Schema:, a fixed, unalterable instruction string that forces the model to synthesize the entire technical audit into a deterministic binary yes or no boolean, encapsulated securely

within <score> tags.⁴² This structural rigidity effectively transforms the large language model into a highly reliable, programmatic logic gate perfectly suited for an automated engineering pipeline.

Evaluating Competing Safety and Audit Architectures

When comparing Granite Guardian against competing safety and alignment methodologies, the fundamental differences in architectural intent become apparent, particularly regarding the ability to parse esoteric engineering compliance.

Meta's Llama Guard 3, available in both 1B and 8B configurations, is an exceptionally potent model fine-tuned specifically for content safety classification.⁴⁸ However, Llama Guard 3 is intrinsically bound to a predefined taxonomy of harm.⁴⁸ It operates by categorizing prompts and responses into strict numerical buckets, such as "S10" or "S12," which correspond to violations like hate speech, sexual content, or defamation.⁴⁸ While the system allows developers to explicitly exclude certain categories from the evaluation by passing an array of category keys, the model is fundamentally optimized for sociological content moderation rather than evaluating the continuous variables of a physics violation log against an FIA technical passport.⁴⁹ Modifying Llama Guard 3 to act as an engineering compliance auditor requires extensive prompt engineering that fights against its core alignment training.

Frontier safety methodologies, such as Anthropic's Constitutional AI (CAI) or the safety stacks developed by OpenAI, rely on entirely different paradigms, utilizing generalized reinforcement learning from human feedback (RLHF) or broad constitutional principles. While models like Claude 4.5 Sonnet and Gemini 2.5 Pro offer unmatched generalized reasoning capabilities—allowing them to deeply understand the semantic nuances of a complex regulatory document—utilizing them as a high-frequency, deterministic audit gate within a real-time racing engineer architecture is wildly inefficient. These models lack the forced binary XML scoring constraints native to Granite Guardian, and their inherent conversational nature makes them prone to conversational hallucination, where they output paragraphs of unnecessary explanation alongside the required boolean score, disrupting the automated software parser.⁴²

Similarly, deploying massive open-weight models such as Llama 3.3 70B Instruct or Mistral Large 2 (123 billion parameters) to perform the audit is computationally untenable.⁵¹ While Mistral Large 2 offers a massive 128k context window that could easily digest the entire FIA rulebook, its massive parameter count prevents it from running on any low-latency edge computing hardware or WebGPU instance, strictly confining it to heavy cloud infrastructure and completely invalidating it for a real-time vehicular deployment where milliseconds dictate physical safety.⁵¹

Evaluation Across Six Core Architectural Dimensions

To definitively ascertain the optimal foundation model stack for the APEX race engineer, the candidates must be systematically evaluated across six critical dimensions specific to the "frozen-backbone + projection + audit" pattern.

1. Sample Efficiency (Small Fine-Tune Data Lifts)

The ability to rapidly adapt the forecasting model to novel sensor arrays with minimal historical data is crucial. Here, the Granite TTM r2.1 model demonstrates remarkable efficiency, requiring only five percent of the target data to adequately adapt its channel-independent backbone via the channel-mixing decoder.¹ However, **Amazon Chronos-2** unequivocally wins this dimension. Because Chronos-2 leverages a group attention mechanism to facilitate zero-shot in-context learning, it natively understands cross-channel dependencies and dynamic covariates without requiring any weight updates or fine-tuning datasets whatsoever, entirely bypassing the adaptation phase required by Granite.⁹ Reprogramming frameworks like Time-LLM and AutoTimes are highly inefficient by comparison, demanding significant computational overhead to force LLM token embeddings into recognizing numerical time-series patches.²⁴

2. Bring-Your-Own-Code (BYOC) Custom-Rule Extensibility

The auditing layer must conform perfectly to arbitrary rules rather than generalized safety taxonomies. **Granite Guardian 4.1 8B** is the definitive winner in this category. Its explicit architectural design for arbitrary, user-defined criteria, combined with its native <guardian> prompt template and forced deterministic <score> tag output, allows developers to map complex multi-part requirements directly into the prompt without fighting the model's alignment.⁴² Llama Guard 3's reliance on excluding predefined string-based taxonomies makes it highly cumbersome for bespoke engineering rules, and frontier models lack the structural rigidity required for flawless programmatic parsing.⁴⁸

3. License and Commercial Freedom

The commercial viability of the APEX architecture is directly tied to the licensing terms of its underlying models. The **IBM Granite Ecosystem**, alongside **Amazon Chronos-2** and **Lag-Llama**, share the victory in this dimension, as they are universally released under the highly permissive Apache 2.0 license, granting unrestricted commercial use, modification, and distribution.⁵⁴ This contrasts sharply with Salesforce Moirai, which is bound by the CC BY-NC 4.0 license, strictly prohibiting commercial deployment without an explicit, negotiated commercial license from Salesforce.⁵⁹ Furthermore, Mistral Large 2 utilizes the Mistral Research License, which permits open usage for research but requires a paid commercial license for any self-deployed commercial integration.⁵¹

4. In-Region Availability (EU / US / Hybrid Cloud)

Compliance with stringent data residency laws requires flexible deployment infrastructure. The **IBM Granite Ecosystem** wins this dimension due to its ubiquitous hybrid cloud availability. Granite models are natively hosted on IBM watsonx.ai, deeply integrated into Oracle Cloud Infrastructure (OCI) via AI Quick Actions, heavily supported on Hugging Face, and entirely capable of being quantized and run locally via Ollama and LM Studio for completely air-gapped sovereign cloud deployments.⁵⁵ While Meta's Llama models enjoy similar open-weight hosting

flexibility, the frontier models from Anthropic (Claude 4.5) and Google (Gemini 2.5) are inherently API-restricted, geographically constraining them to the specific infrastructure rollouts of their parent companies and preventing true air-gapped hybrid autonomy.

5. Edge and WebGPU Compatibility

Vehicular telemetry demands absolute minimum latency, requiring inference to occur directly on edge compute units or within lightweight in-browser WebGPU environments. The **Granite TimeSeries** suite (specifically TSPulse and TTM) completely dominates this dimension. Operating at a microscopic scale of just one million parameters, these models are ultra-compact and specifically designed to support GPU-free inference, allowing them to run effortlessly on the constrained microcontrollers inherent to automotive engineering.¹ While Google TimesFM's 200 million parameters offer reasonable efficiency, it remains heavily reliant on PyTorch GPU acceleration.¹³ The concept of deploying Mistral Large 2 (123B) or Llama 3.3 (70B) in an edge environment is entirely unfeasible due to their massive VRAM requirements.⁵¹

6. Long-Tail Safety and Niche Regulatory Parsing

Standard safety models are oblivious to the localized legal realities of the adaptive racing niche. When evaluating whether a driver's simultaneous use of a hand-throttle and foot-brake violates the mathematical projection envelope, the audit layer must cross-reference the highly specific permissions outlined in an FIA Certificate of Adaptation (e.g., Regulation D-050).³⁷ **Granite Guardian 4.1 8B** emerges as the victor, as its rigid BYOC framework is perfectly tailored to ingest complex legal and technical dossiers and execute a targeted, deterministic binary audit against the continuous-variable outputs of the physics projection, a task where broad Constitutional AI principles fundamentally fail to understand the localized nuance of the de-homologation.⁴⁰

Architectural Dimension	Granite Ecosystem Capability	Primary Competitor Strategy	Dimension Winner
Sample Efficiency	5% Target Data for Channel-Mixing	Chronos-2 Zero-Shot In-Context Learning	Amazon Chronos-2
BYOC Extensibility	Native XML-gated Custom Criteria	Llama Guard 3 Taxonomy Exclusion	Granite Guardian 4.1
Commercial License	Permissive Apache 2.0	Salesforce Moirai CC BY-NC 4.0	Granite / Chronos / Lag-Llama
Hybrid Availability	watsonx, OCI,	Claude / Gemini	IBM Granite

	Air-Gapped Local	API-Locked Architecture	Ecosystem
Edge Compute	1M to 9.1M Parameter Range	Google TimesFM 200M Range	Granite TimeSeries
Long-Tail Safety	Deterministic Dossier Parsing	Anthropic CAI Broad Principles	Granite Guardian 4.1

The Defensible Advantages of the Granite-Exclusive Architecture

For the specific APEX submission to the IBM SkillsBuild Challenge, committing entirely to the IBM Granite foundation models yields three highly defensible, structurally sound architectural advantages that justify its selection over competing frameworks.

1. Extreme Micro-Parameter Viability for Local Edge Execution

The defining competitive advantage of the Granite TimeSeries suite (TTM, FlowState, TSPulse) is its extreme parameter efficiency. By deliberately restricting the model size to a scale between 1 million and 9.1 million parameters, the Granite architecture completely bypasses the severe latency, power consumption, and thermal constraints associated with vehicular edge computing.¹ In stark contrast to the Time-LLM framework, which necessitates the deployment of a multi-billion parameter LLM simply to generate standard forecasting outputs, or Amazon Chronos-2, which relies on vastly heavier T5-equivalent architectures, the Granite suite enables the APEX system to run GPU-free, synchronous forecast generation natively. This ultra-lightweight footprint ensures that the highly computationally demanding operations of the downstream cvxpylayers physics projection are not starved of memory or compute, maintaining the integrity of the real-time control loop.¹

2. Deterministic BYOC Auditing for Idiosyncratic Legal Compliance

Standard AI safety architectures and guardrails are fundamentally designed to mitigate broad sociotechnical harms across massive populations.⁴⁸ However, the adaptive racing environment requires the arbitration of hyper-specific, highly localized physical compliance documents—namely, the individual FIA Certificates of Adaptation.³⁸ Granite Guardian 4.1 is uniquely purpose-built to execute arbitrary, user-defined criteria via a fixed prompt schema designed to generate a machine-parsable <score> tag.⁴² This structural rigidity provides the APEX architecture with a deterministically reliable, programmatic audit gate. Competing platforms, such as Llama Guard 3 or Anthropic's Constitutional AI, cannot natively replicate this deterministic boolean parsing without extensive, highly fragile prompt wrapper engineering

that constantly risks conversational hallucination.⁴²

3. Unified, Frictionless Commercial Deployment via Apache 2.0

The entire IBM Granite ecosystem utilized in the APEX architecture—spanning from the time-series forecasting predictors to the Guardian textual audit layers—is uniformly released under the permissive Apache 2.0 license.¹ This cohesive licensing strategy completely eliminates the legal fragmentation and compliance risks common in mixed-model architectures. Competitors in the space frequently impose highly restrictive or ambiguous terms; Salesforce Moirai explicitly prohibits commercial deployment without a paid license agreement under its CC BY-NC 4.0 terms⁵⁹, and Mistral Large 2 restricts self-deployed commercialization efforts under the terms of the Mistral Research License.⁵¹ By standardizing on Granite, the APEX platform guarantees total commercial freedom, code sovereignty, and unencumbered scalability for future iterations.

Concessions and Areas Where the Granite Stack Lags Industry Leaders

While the Granite stack is highly optimized for efficiency, deterministic parsing, and commercial freedom, the engineering decision to utilize an exclusively Granite-based architecture requires accepting definitive operational trade-offs. The stack currently lags behind industry leaders in three specific technical domains.

1. Zero-Shot Contextual Adaptation for Multivariate Telemetry

While Granite TTM r2.1 provides excellent computational efficiency, its core architecture relies on a channel-independent backbone during pretraining.³ To adequately understand the deep cross-channel interactions critical to complex vehicle dynamics—such as the mathematical relationship between lateral G-forces and steering angle input under heavy braking—TTM explicitly requires fine-tuning the decoder module with approximately five percent of the target dataset.¹ In stark contrast, Amazon Chronos-2 utilizes a highly sophisticated group attention mechanism that achieves vastly superior zero-shot in-context learning.⁹ Chronos-2 seamlessly handles complex multivariate relationships and dynamic exogenous covariates directly out-of-the-box without requiring any secondary fine-tuning pipeline.¹² The APEX architecture deliberately accepts this zero-shot inferiority to maintain the sub-10M parameter footprint of Granite, a scale that Chronos-2 cannot mathematically match.

2. Deep Epistemic Uncertainty Quantification

Operating at the extreme limits of high-speed kinematics fundamentally requires highly robust epistemic uncertainty quantification; standard point forecasts are dangerously insufficient when a vehicle is dynamically balanced on the absolute edge of a friction ellipse. While IBM provides extensive post-hoc conformal prediction pipelines and adaptive anomaly detection strategies within the tsfm ecosystem to address this⁶⁶, the Granite TimeSeries models primarily optimize for deterministic point predictions. Conversely, Salesforce Moirai natively

maps its outputs directly to mixture distributions¹⁷, and the Lag-Llama framework intrinsically outputs the core mathematical parameters (mean, scale, degrees of freedom) of a heavy-tailed Student's t-distribution to estimate uncertainty directly within the neural network.¹⁹ Google TimesFM also natively produces quantile forecasts without external wrappers.¹⁴ The Granite stack requires the integration of secondary algorithmic wrappers to achieve the comprehensive probabilistic bounds that its competitors generate natively, introducing additional computational steps into the pipeline.

3. Deep Logical Reasoning in the Textual Audit Layer

Granite Guardian 4.1 8B excels at following strict, highly formatted BYOC binary rubrics and executing them with incredibly low latency. However, when the physics violation log generated by cvxpylayers contains deeply ambiguous interactions, or requires highly complex multi-step logical deduction to correlate a specific physical anomaly with a dense, 50-page FIA regulatory dossier, the 8B parameter model inevitably hits a reasoning ceiling. Massive frontier models, such as Claude 4.5 Sonnet, Gemini 2.5 Pro, and Llama 3.3 70B Instruct, possess vastly superior internal world models, expansive context windows, and profound logical reasoning depths.⁵¹ The APEX platform deliberately sacrifices this deep, generalized reasoning capacity in exchange for Granite Guardian's strict structural XML compliance and significantly lower inference costs. This trade-off is accepted under the architectural assumption that the continuous-variable output of the cvxpylayers projection layer will be formatted cleanly and rigidly enough to completely bypass the need for the deep semantic deduction capabilities of frontier models.

Works cited

1. ibm-granite/granite-timeseries-ttm-r2 - Hugging Face, accessed May 25, 2026, <https://huggingface.co/ibm-granite/granite-timeseries-ttm-r2>
2. Granite Time Series - IBM Granite, accessed May 25, 2026, <https://www.ibm.com/granite/docs/models/time-series>
3. Fine-tune Granite Time Series - IBM, accessed May 25, 2026, <https://www.ibm.com/granite/docs/fine-tune/time-series>
4. Open-source Granite TimeSeries tiny model | by Alain Airom (Ayrom) | Medium, accessed May 25, 2026, <https://alain-airom.medium.com/open-source-granite-timeseries-tiny-model-9dd6dfd807b3>
5. ibm-granite/granite-timeseries-flowstate-r1 - Hugging Face, accessed May 25, 2026, <https://huggingface.co/ibm-granite/granite-timeseries-flowstate-r1>
6. FlowState: Sampling Rate Invariant Time Series Forecasting - arXiv, accessed May 25, 2026, <https://arxiv.org/html/2508.05287v1>
7. ibm-granite/granite-timeseries-tspulse-r1 - Hugging Face, accessed May 25, 2026, <https://huggingface.co/ibm-granite/granite-timeseries-tspulse-r1>
8. [2505.13033] TSPulse: Tiny Pre-Trained Models with Disentangled Representations for Rapid Time-Series Analysis - arXiv, accessed May 25, 2026, <https://arxiv.org/abs/2505.13033>

9. Introducing Chronos-2: From univariate to universal forecasting - Amazon Science, accessed May 25, 2026, <https://www.amazon.science/blog/introducing-chronos-2-from-univariate-to-universal-forecasting>
10. amazon-science/chronos-forecasting: Chronos: Pretrained Models for Time Series Forecasting - GitHub, accessed May 25, 2026, <https://github.com/amazon-science/chronos-forecasting>
11. amazon/chronos-2 - Hugging Face, accessed May 25, 2026, <https://huggingface.co/amazon/chronos-2>
12. [2510.15821] Chronos-2: From Univariate to Universal Forecasting - arXiv, accessed May 25, 2026, <https://arxiv.org/abs/2510.15821>
13. A decoder-only foundation model for time-series forecasting - Google Research, accessed May 25, 2026, <https://research.google/blog/a-decoder-only-foundation-model-for-time-series-forecasting/>
14. google/timesfm-2.5-200m-transformers - Hugging Face, accessed May 25, 2026, <https://huggingface.co/google/timesfm-2.5-200m-transformers>
15. TimesFM 2.5 - Hugging Face, accessed May 25, 2026, https://huggingface.co/docs/transformers/model_doc/timesfm2_5
16. TimesFM - Hugging Face, accessed May 25, 2026, https://huggingface.co/docs/transformers/model_doc/timesfm
17. Moirai: A Time Series Foundation Model for Universal Forecasting - Salesforce, accessed May 25, 2026, <https://www.salesforce.com/blog/moirai/>
18. [2310.08278] Lag-Llama: Towards Foundation Models for Probabilistic Time Series Forecasting - arXiv, accessed May 25, 2026, <https://arxiv.org/abs/2310.08278>
19. Lag-Llama: Towards Foundation Models for Probabilistic Time Series Forecasting - GitHub Pages, accessed May 25, 2026, <https://time-series-foundation-models.github.io/lag-llama.pdf>
20. Lag-Llama: Unifying Time-Series Forecasting with Foundation Model Principles - Medium, accessed May 25, 2026, <https://medium.com/@kdk199604/lag-llama-unifying-time-series-forecasting-with-foundation-model-principles-c6bf4e0dbb26>
21. [ICLR 2024] Official implementation of "Time-LLM: Time Series Forecasting by Reprogramming Large Language Models" - GitHub, accessed May 25, 2026, <https://github.com/KimMeen/Time-LLM>
22. [2310.01728] Time-LLM: Time Series Forecasting by Reprogramming Large Language Models - arXiv, accessed May 25, 2026, <https://arxiv.org/abs/2310.01728>
23. Time-LLM: Time Series Forecasting by Reprogramming Large Language Models, accessed May 25, 2026, <https://openreview.net/forum?id=Unb5CVptae>
24. AutoTimes: Autoregressive Time Series Forecasters via Large Language Models, accessed May 25, 2026, <https://neurips.cc/virtual/2024/poster/95975>
25. Neural QAOA 2 : Differentiable Joint Graph Partitioning and Parameter Initialization for Quantum Combinatorial Optimization - arXiv, accessed May 25, 2026, <https://arxiv.org/html/2605.13072v1>
26. Enforcing Policy Feasibility Constraints through Differentiable Projection for

- Energy Optimization - arXiv, accessed May 25, 2026,
<https://arxiv.org/pdf/2105.08881>
27. GitHub - pnnl/neuromancer: Pytorch-based framework for solving parametric constrained optimization problems, physics-informed system identification, and parametric model predictive control., accessed May 25, 2026,
<https://github.com/pnnl/neuromancer>
 28. Efficient and Modular Implicit Differentiation - Semantic Scholar, accessed May 25, 2026,
<https://pdfs.semanticscholar.org/90ae/ef430008dbb1c14667f70cf7a0887932c495.pdf>
 29. Pinet: Optimizing hard-constrained neural networks with orthogonal projection layers, accessed May 25, 2026, <https://openreview.net/forum?id=EJ680UQeZG>
 30. Chapter 5: Differentiable optimization - Deep Implicit Layers, accessed May 25, 2026, http://implicit-layers-tutorial.org/differentiable_optimization/
 31. Implicit Differentiation for Fast Hyperparameter Selection in Non-Smooth Convex Learning, accessed May 25, 2026,
<https://www.jmlr.org/papers/volume23/21-0486/21-0486.pdf>
 32. Fast Differentiable Projection Layers onto High-Dimensional Polytopes for Large-Scale Predictive Modeling - ucf stars, accessed May 25, 2026,
<https://stars.library.ucf.edu/cgi/viewcontent.cgi?article=1498&context=etd2024>
 33. Solvers - CVXPYlayers, accessed May 25, 2026,
<https://cvxpylayers.org/guide/solvers.html>
 34. Circle Packing problem with CVXPYlayers - Operations Research Stack Exchange, accessed May 25, 2026,
<https://or.stackexchange.com/questions/13567/circle-packing-problem-with-cvxpylayers>
 35. possible to ignore Solved/Inaccurate ? · Issue #82 · cvxpy/cvxpylayers - GitHub, accessed May 25, 2026, <https://github.com/cvxpy/cvxpylayers/issues/82>
 36. "Nonsmooth Implicit Differentiation for Machine Learning and Optimization" - Toulouse School of Economics, accessed May 25, 2026,
https://www.tse-fr.eu/sites/default/files/TSE/documents/doc/wp/2022/wp_tse_1314.pdf
 37. Defying all odds - People Daily, accessed May 25, 2026,
<https://peopledaily.digital/sports/defying-all-odds>
 38. disability and accessibility commission - fia - Guidosimplex.it, accessed May 25, 2026,
https://www.guidosimplex.it/eventi/fia/Certificate-2024_Ford_Llovera_Rally-Raid.pdf
 39. Differently abled in karting. Where are we at?, accessed May 25, 2026,
<https://www.vroomkart.com/news/40865/differently-abled-in-karting-where-are-we-at>
 40. FIA Certificate of Adaptations, accessed May 25, 2026,
https://www.fia.com/sites/default/files/coa_iveco_powerstar_4x4_2021.pdf
 41. Communicating the Condition of Terrestrial Ecosystems a Focused Investigation of Terrestrial Ecosystem Health Indicators - epa nepis, accessed May 25, 2026,

- <https://nepis.epa.gov/Exe/ZyPURL.cgi?Dockkey=91023ZCP.TXT>
42. ibm-granite/granite-guardian: The Granite Guardian models ... - GitHub, accessed May 25, 2026, <https://github.com/ibm-granite/granite-guardian>
 43. ibm-granite/granite-guardian-4.1-8b - Hugging Face, accessed May 25, 2026, <https://huggingface.co/ibm-granite/granite-guardian-4.1-8b>
 44. Build an agentic AI safety pipeline with Runpod Flash and Granite Guardian 4.1, accessed May 25, 2026, <https://www.runpod.io/blog/building-agentic-safety-checks-with-runpod-flash-and-ibm-granite-4-1>
 45. D-050 - Aquestia, accessed May 25, 2026, https://www.aquestia.com/wp-content/uploads/2024/11/ARI.D050.IND_.CAT_.EN_.1124.pdf
 46. Granite Guardian 4.1 - IBM, accessed May 25, 2026, <https://www.ibm.com/granite/docs/models/guardian>
 47. IBM watsonx.ai Chat and Generation Models - Quarkiverse Documentation, accessed May 25, 2026, <https://docs.quarkiverse.io/quarkus-langchain4j/dev/watsonx-chat-model.html>
 48. Block unsafe prompts targeting your LLM endpoints with Firewall for AI - The Cloudflare Blog, accessed May 25, 2026, <https://blog.cloudflare.com/block-unsafe-llm-prompts-with-firewall-for-ai/>
 49. meta-llama/Llama-Guard-3-1B - Hugging Face, accessed May 25, 2026, <https://huggingface.co/meta-llama/Llama-Guard-3-1B>
 50. Guardrails: Enterprise safety shields with Llama Stack | Red Hat Developer, accessed May 25, 2026, <https://developers.redhat.com/articles/2026/05/04/guardrails-enterprise-safety-shields-llama-stack>
 51. Large Enough | Mistral AI, accessed May 25, 2026, <https://mistral.ai/news/mistral-large-2407>
 52. Granite 3.3 8B Base vs Mistral Large 2 Comparison - LLM Stats, accessed May 25, 2026, <https://llm-stats.com/models/compare/granite-3.3-8b-base-vs-mistral-large-2-2407>
 53. mistral-large/license - Ollama, accessed May 25, 2026, <https://ollama.com/library/mistral-large:latest/blobs/9bac90050d38>
 54. IBM Granite TimeSeries TTM - AWS Marketplace, accessed May 25, 2026, <https://aws.amazon.com/marketplace/pp/prodview-az47mhw6aye2k>
 55. IBM Granite 3.2: open source reasoning and vision, accessed May 25, 2026, <https://www.ibm.com/new/announcements/ibm-granite-3-2-open-source-reasoning-and-vision>
 56. Introducing IBM Granite 3.3 Model on OCI Data Science | ai-and-datascience - Oracle Blogs, accessed May 25, 2026, <https://blogs.oracle.com/ai-and-datascience/introducing-ibm-granite-33>
 57. lag-llama/LICENSE at main - GitHub, accessed May 25, 2026, <https://github.com/time-series-foundation-models/lag-llama/blob/main/LICENSE>
 58. README.md · time-series-foundation-models/Lag-Llama at

- 9036517c3d9d2ebb42cb265b160d7092791619f0 - Hugging Face, accessed May 25, 2026,
<https://huggingface.co/time-series-foundation-models/Lag-Llama/blame/9036517c3d9d2ebb42cb265b160d7092791619f0/README.md>
59. Deploy Time Series Foundation Models on GPU Cloud: Chronos, Moirai, TimesFM, and Lag-Llama Production Setup Guide (2026) | Spheron Blog, accessed May 25, 2026,
<https://www.spheron.network/blog/deploy-time-series-foundation-models-gpu-cloud/>
60. Salesforce/moirai-1.0-R-large - Hugging Face, accessed May 25, 2026,
<https://huggingface.co/Salesforce/moirai-1.0-R-large>
61. Mistral AI's next-generation flagship LLM, Mistral Large 2, is now available in IBM watsonx™, accessed May 25, 2026,
<https://www.ibm.com/new/announcements/mistral-ais-next-generation-flagship-llm-mistral-large-2-is-now-available-in-ibm-watsonx>
62. What Is Mistral Large 2? How It Works, Use Cases & More - DataCamp, accessed May 25, 2026,
<https://www.datacamp.com/blog/mistral-large-2>
63. Introducing the IBM Granite 4.1 family of models, accessed May 25, 2026,
<https://research.ibm.com/blog/granite-4-1-ai-foundation-models>
64. IBM foundation models, accessed May 25, 2026,
<https://www.ibm.com/docs/en/watsonx/w-and-w/2.3.x?topic=models-foundation>
65. GraniteGuardian | any-guardrail | Mozilla.ai Docs, accessed May 25, 2026,
<https://docs.mozilla.ai/any-guardrail/api-reference/index/granite-guardian>
66. Track: San Diego Poster Session 4 - NeurIPS 2026, accessed May 25, 2026,
<https://neurips.cc/virtual/2025/loc/san-diego/session/128334>
67. Adaptive Conformal Anomaly Detection with Time Series Foundation Models for Signal Monitoring - arXiv, accessed May 25, 2026,
<https://arxiv.org/html/2604.20122v1>
68. granite-tsfm/tests/toolkit/test_time_series_forecasting_pipeline.py at main - GitHub, accessed May 25, 2026,
https://github.com/ibm-granite/granite-tsfm/blob/main/tests/toolkit/test_time_series_forecasting_pipeline.py